

南京大学《生成式软件工程》2026 秋季学期

第 1 讲：欢迎来到未来

AI、Agent 与软件工程的重新定位

主讲：蒋炎岩 中文精讲笔记



视频标题：欢迎来到未来 [01-Raw/26 生成式软件工程/NJU]

视频作者/频道：绿导师原谅你了

发布日期：2026-08-27

时长：01:39:59

视频：<https://www.bilibili.com/video/BV1pb8o6yE8f/>

课程主页：<https://jyywiki.cn/GSE/2026/>

目录

1 导读：这不是一堂工具速成课	2
2 交叉路口：能力日用品化与学习危机	3
3 课程的答案：可复现的开放工程	7
4 从 LLM 到 Agent：上下文成为可写状态	10
5 Agent 案例：把事情真正推进	16
6 从白板 Agent 到完整生态	18
7 现场造工具：边界、协议与证据链	24
8 qrgen 失败案例：跑出来不等于做成	28
9 软件工程为什么仍然必要	33
10 总结与延伸	39

1 导读：这不是一堂工具速成课

本讲主问题

当代码生成、搜索、工具调用和长程任务执行逐渐成为廉价能力以后，计算机专业学生还需要学习什么？软件工程又应该保护什么？

本讲既是课程说明，也是一次软件工程立场的重建。讲者没有把 Agent 描述成一个更聪明的聊天框，而是把它放入真实系统：它要读取信息、修改状态、调用工具、观察结果、发现错误、重新规划，并最终对可验证的结果负责。由此，问题从“模型能不能写代码”升级为“人和工具怎样把一件大事可靠地做成”。

本笔记使用四类证据：Bilibili 原视频、Whisper 生成并校订的时间轴字幕、每 15 秒抽取的完整画面，以及南京大学课程主页。正文中的 42 张图均经过联系表筛选、全分辨率复核和字幕上下文核对；图下注明的 15 秒区间可直接回到原视频。课程中的玩笑和即兴表达只在帮助理解时保留，所有课程管理信息都被还原为其背后的设计理由。

贯穿全讲的四条线索

1. **能力日用品化**：PC、互联网、移动互联网和 AI 都把曾经昂贵的能力降到大众可用。
2. **Agent 化**：系统不再只返回文本，而是把上下文、工具和可观察环境连成反馈循环。
3. **Everything Is Code**：文本、视频、论文、表格、协议乃至制造描述，都可以成为可重放、可修改、可验证的计算资产。
4. **软件工程不死**：失败仍然是系统没有满足真实意图与约束；只是实现速度提高以后，失败更容易被快速放大。

1.1 阅读方法

建议每个案例都问四个问题：输入和状态是什么？Agent 能做哪些动作？完成条件是否可观察？失败后能否以低成本回滚或缩小问题？若缺少其中任意一项，所谓“自动完成”通常只是一次无法复用的演示。

1.2 全讲路线图

视频区间	主要问题
00:00–07:00	能力为何突然成为日用品；AI 代做以后，作业和简历还能证明什么
07:00–20:30	AI 时代的软件工程智慧、开放课程组织、Token 与通过政策
20:30–32:15	LLM 的 next-token 接口、知识重编码、ChatGPT 到 Agent、可写 context
32:15–50:30	Debian、邮件和论文阅读三个 Agent 案例
50:30–65:15	Agent 生态、Computer Use、CDP、轨迹复用和 Harness 可靠性
65:15–74:45	考试统分、隔离机二维码通道与 Everything Is Code
74:45–89:00	qrngen 失败、系统成功率、规格空间、架构转折、Intent/Spec/Impl
89:00–99:59	gap、软件工程研究、风险退休、人的上游责任

1.3 本章小结

课程的中心不是追逐某个模型或产品，而是学习把开放世界中的模糊意图压缩成可执行、可观察、可验证、可追责的工程闭环。

2 交叉路口：能力日用品化与学习危机

2.1 四次能力下放

Altair 8800 代表个人计算，Mosaic 代表大众互联网，iPhone 代表移动互联网，ChatGPT 代表生成式 AI。四次转折的共同点并不是机器只强了一点，而是某种过去稀缺的能力突然成为日用品。日用品化改变的不只是生产效率，也会改变课程、职业和组织的默认分工。

四次把昂贵能力变成日用品

PC (1975, Altair 8800)、互联网 (1993, Mosaic)、移动互联网 (2007, iPhone)、AI (2022, ChatGPT), 在人类命运的交叉路口, 需要一个非常实验性的课程。



✦ 真正的拐点, 不是机器又强了一点; 而是某种昂贵能力, 突然变成了日用品。

9/9

图 1: 四次把昂贵能力变成日用品: PC、互联网、移动互联网与生成式 AI¹

交叉路口的真正含义

旧任务可以更快完成, 不等于旧训练仍能形成能力。教育必须重新设计反馈、作品和责任, 使学生不能只把输入交给模型、再把输出交给教师。

2.2 作业完成不等于学习发生

过去, 作品通常是能力留下的痕迹: 程序能运行, 意味着学生至少亲自经历过分解、调试和修正。生成式工具打破了这一推断。同一份正确作业可能来自完全不同的能力, 因此“通过测试”只能证明产物满足部分显式条件, 不能直接证明学生掌握了问题。

¹视频画面时间区间: 00:00:00–00:00:15。

Why? 狼来了

作业完成了，学习发生了吗？

Vibe coding 很厉害，作业直接丢进 codex 就完成了。

◆ 过去，作品 = 能力留下的痕迹；现在，同一份作品可能对应完全不同的能力。

“

◆ 如果 Codex 能交出满分作业，满分究竟证明了谁的什么能力？

2 / 3

图 2: 作业被 Codex 完成之后，学习是否真正发生²

这并不推出“禁止 AI”。更合理的课程设计是把学习证据从静态答案移向可追溯过程：学生能否解释关键决策，能否构造反例，能否定位失败，能否在约束改变时维护作品。

2.3 三种焦虑来自同一个根

第一种焦虑是把学习外包给 AI，毕业时发现自己没有形成能力；第二种焦虑是认真完成旧课程，却发现所学任务已经被自动化；第三种焦虑是简历和作品都被模型生成，人与人之间失去可辨认的信号。三者的共同根源是把“完成任务”误认为“形成稀缺能力”。

²视频画面时间区间：00:02:15–00:02:30。

三种焦虑

三个问题，一个共同的根

- ◆ AI 正在把“学习”、“文凭”和“被选中”拆成三件事：
- 焦虑 (1): AI 帮我上大学，最后什么都没学会。怎么办？
- 焦虑 (2): 听话学习了四年，发现被大学骗了？
- 焦虑 (3): 为了刷出一份完美的简历，输了人生？
- ◆ 我是在增长能力，还是在增长“看起来有能力”的证据？

图 3: AI 帮上大学、大学所学被替代与简历同质化的三种焦虑³

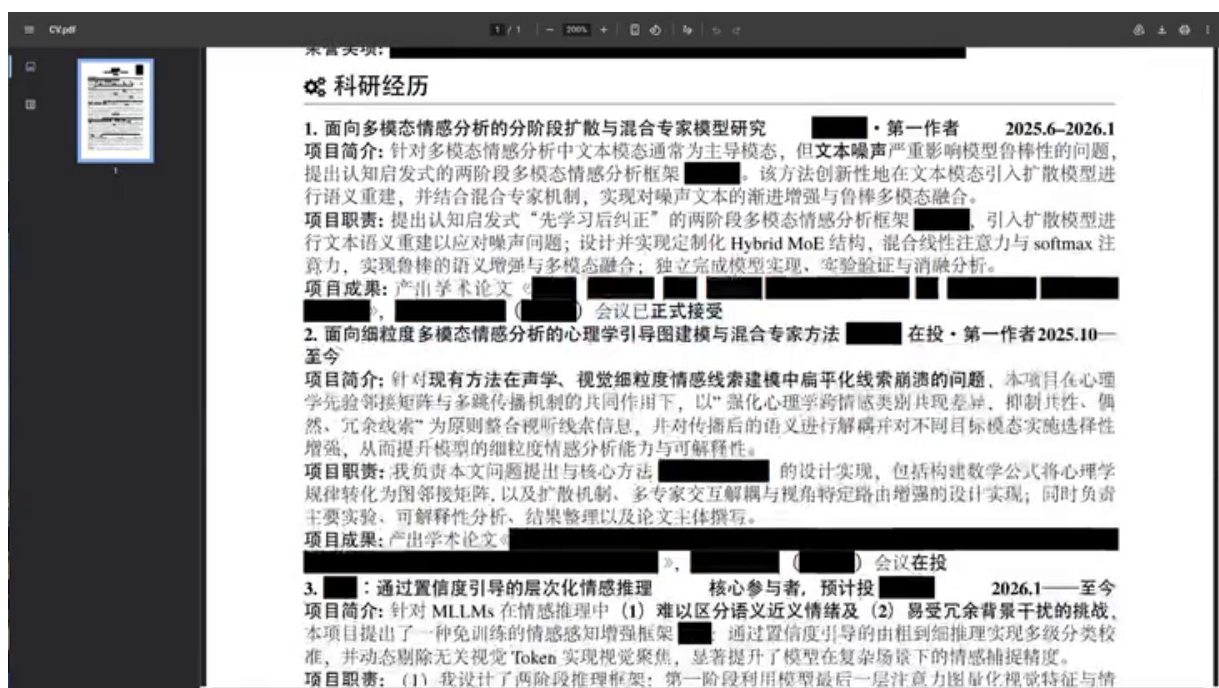


图 4: 一份高度同质化简历及其缺失的可验证能力信号⁴

一份表面漂亮但无法追溯的简历缺少的不是更多关键词，而是可验证的个人痕迹：公开代码、持续演化的项目、设计取舍、错误记录和他人可复现的结果。在作品普遍可以生成以后，证据链本身成为能力的一部分。

³视频画面时间区间：00:03:30–00:03:45。

⁴视频画面时间区间：00:05:15–00:05:30。

不要把焦虑转化为错误结论

“AI 能完成我的作业”既不意味着学习无用，也不意味着必须拒绝 AI。它意味着考核单位需要改变：从一次提交，变为问题选择、过程证据、验证能力、维护成本和最终责任。

2.4 本章小结

能力日用品化使旧产物失去鉴别力。新的学习目标不是和模型比手速，而是获得提出问题、驾驭工具、识别风险和证明结果的能力。

3 课程的答案：可复现的开放工程

3.1 软件工程研究的是怎样不失败

讲者把软件工程的目标概括为“让工程不失败”。AI 可以加速实现，却不会自动暴露未知约束、分配责任或验证端到端结果。系统必须在多人协作、环境变化和局部故障中继续活着，因此工程能力更接近驾驭：让人、Agent 和反馈持续对齐目标。

What? AI 时代还有用的“软件工程智慧”

软件工程研究的是“怎么让工程不失败”

- ◆ AI 可以加速实现，却不会自动暴露未知、分配责任或验证结果
- ◆ 工程的目标不是让 demo 跑起来，而是让系统在约束、变化和协作中继续活着

大学还能教你的

- 信息差、能坐牢、**能驾驭**
- ◆ 信息差：知道哪里不知道；能坐牢：为取舍承担后果；能驾驭：让人、Agent 与反馈持续对齐目标
- ◆ Mars Climate Orbiter 的单位接口事故提醒我们：风险常发生在团队边界；真正的工程能力，是建立能在发射前叫停错误的闭环

1/6

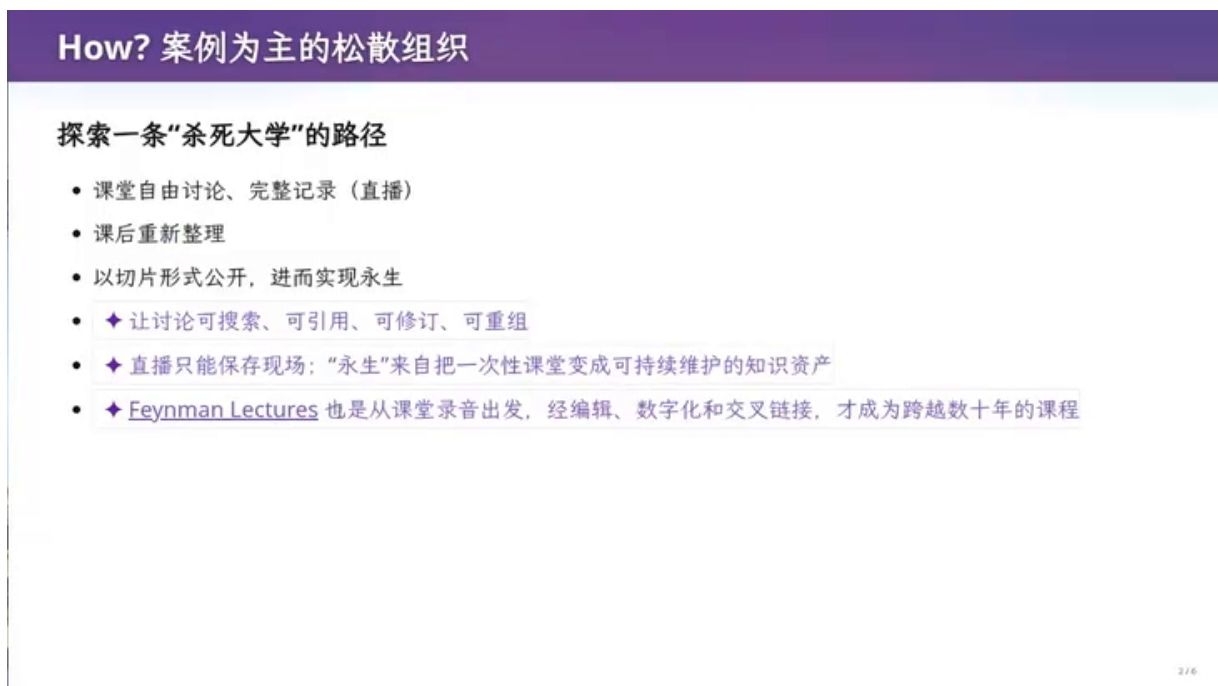
图 5: AI 时代仍有用的软件工程智慧：信息差、能坐牢与能驾驭⁵

其中“信息差、能坐牢、能驾驭”是一组带有课堂幽默的压缩表达。信息差指知道哪里不知道；“能坐牢”强调法律与社会责任最终落在人身上；能驾驭则是把目标、分工、验证和纠错组织成闭环。

⁵视频画面时间区间：00:07:00–00:07:15。

3.2 案例、公开过程与可再生讲义

课程采用案例为主的松散组织：课堂允许讨论，直播保留完整过程，课后再重整为公开材料。这里的“永生”不是把一次讲课录像永久保存，而是保留可编辑、可 diff、可重建的源文件和生成流程。



How? 案例为主的松散组织

探索一条“杀死大学”的路径

- 课堂自由讨论、完整记录（直播）
- 课后重新整理
- 以切片形式公开，进而实现永生
- ✦ 让讨论可搜索、可引用、可修订、可重组
- ✦ 直播只能保存现场：“永生”来自把一次性课堂变成可持续维护的知识资产
- ✦ [Feynman Lectures](#) 也是从课堂录音出发，经编辑、数字化和交叉链接，才成为跨越数十年的课程

2 / 6

图 6: 案例为主、课后重整与公开可复现的课程组织方式⁶



Everything is code

这套 slides 如何工作？

- 文本是 source of truth；你看到的其他模式都是 AIGC
- ✦ 完整讲义提供上下文 → 选中的 snippet 划定边界 → Agent 扩写 → Markdown slides → 浏览器渲染
- ✦ 生成内容统一标记为 aigc；逻辑改变时修改源码，图片、网页和视频切片都可以重新生成

要“永生”的不是一份录像

- ✦ 保存可编辑、可 diff、可复现的源代码与构建过程，而不只是最终像素
- 如果成功，就“永生”我的操作系统课

3 / 6

图 7: 以文本为 source of truth 的幻灯片生成与可再生 workflow⁷

⁶视频画面时间区间：00:08:45–00:09:00。

⁷视频画面时间区间：00:13:00–00:13:15。

幻灯片 workflow 把人工写下的文本当作 source of truth，再由 Agent 补全上下文、重写选中片段、生成 Markdown slides 并交给浏览器渲染。图片和网页片段只是可再生的派生产物。这个例子提前展示了本讲的核心：把内容生产从一次性结果变成可维护系统。

3.3 课程政策是实验设计

Token 自费并不是鼓励无节制消费。自费使成本、质量和延迟成为可感知变量，学生需要知道何时使用更强模型、何时缩小上下文、何时复用结果。真正稀缺的不是 token，而是值得花 token 的问题、清晰约束和可验证实验。

课程 Policy: Token 自费

没有 token 的 CS 学生应该**立即退学**

- 个人观点
- 不需要 tokenmaxxing
- 课程课堂全程 deepseek-v4-flash
- ♦ Token 是 AI 时代的实验耗材：亲自使用，才能形成成本—质量—延迟意识
- ♦ 真正稀缺的不是 token，而是值得花 token 的问题、清晰约束和可靠验收



梁文峰

图 8: Token 自费：把成本、质量与延迟变成实验意识⁸

课程尽量采用通过/不通过，不验收传统作业，也不以分数制造排名。学生需要用一个人可以打开、运行、质疑和继续使用的作品回答“你做成了什么”。在 AI 时代，公开项目历史比孤立分数更接近可审计的成长轨迹。

⁸视频画面时间区间：00:15:30–00:15:45。

课程 Policy：尽量“通过 / 不通过”

不验收作业，不评分

- 有建议的实验和反馈
- 现在越来越难找到客观衡量标准
- “服从性考试没有意义”——大家都一样，职场见
- ✦ 不评分 ≠ 不评价：去掉排名信号，把反馈留给下一次迭代

用作品回答“你做成了什么？”

- 希望大家能在简历里写：“在《生成式软件工程》课程中实现了 A, B, C, D, ... (github.io 链接)”
- 例子：[我的主页](#)
- ✦ 最终证据不是分数，而是别人可以打开、运行、质疑并继续使用的作品；github.io 是一条可审计的成长轨迹

6 / 6

图 9：用作品回答做成了什么，而不是用分数回答⁹

推荐的课程作品证据

- 明确的问题与不做什么；
- 可执行的仓库、部署地址或复现实验；
- 关键失败、回滚和设计转折的版本历史；
- 自动检查与人工判断的边界；
- 后续使用者能否在不询问作者的情况下继续维护。

3.4 本章小结

这门课把教学本身设计成软件工程：源文件公开、过程可追溯、结果可再生、作品可检验，并用真实成本迫使学习者形成实验判断。

4 从 LLM 到 Agent：上下文成为可写状态

4.1 简单接口为何产生复杂行为

自回归语言模型可以写成：

$$P(x_{t+1} \mid x_{\leq t})$$

⁹视频画面时间区间：00:17:30–00:17:45。

- $x_{\leq t}$ 是当前上下文中的 token 序列;
- x_{t+1} 是下一 token;
- 概率分布只保证在给定上下文下的流畅延续, 不直接等价于事实、证明或完成任务。

LLM 和 Agent

一个简单到离谱的接口

LLM (autoregressive): 一个概率分布 $\text{Pr}[\text{token} \mid \text{context}]$

- 它很大、训练不崩很难
- 这门课是“使用者视角”, 所以不需要任何 LLM 的知识

◆ “预测下一个 token”是训练接口, 不是能力上限: 要同时预测代码、证明、对话和故事, 模型就被迫压缩语言背后的世界规律。

- 自回归把静态分布展开成计算过程: 每个输出都会写回 context; 足够长的 token 序列可以承载草稿、算法和推理。

◆ 使用者最有用的心智模型: 它不是答案数据库, 而是一个由 context 条件化的模拟器——我们真正“编程”的是 context。

“ ◆ 流畅, 只说明“在这个上下文里像一个答案”; 它不是正确性的证据。

1 / 4

图 10: LLM 与 Agent 之间的离谱但关键接口: $\text{Pr}(\text{token} \mid \text{context})$ ¹⁰

这个接口看似离谱: 模型只预测下一个 token, 却能生成代码、证明、对话和计划。关键是自回归过程不断把输出写回上下文, 足够长的序列可以承载算法和推理。用户真正“编程”的对象不是答案数据库, 而是由上下文条件化的模拟器。

4.2 知识重编码比背诵更重要

GPT-4 为五岁孩子解释扫描电子显微镜的例子说明, 模型的价值不只是复述术语, 而是根据受众重组知识: 用铅笔扫描纸面、电子反弹和信号成像建立可理解的因果链。知识没有被简化为更强相机, 而是被重新编码为适合当前认知背景的模式。

¹⁰视频画面时间区间: 00:20:30–00:20:45。

GPT-4 (2023): 给五岁孩子解释 SEM

不只是知道，而是为听众重编码

为我的五岁小朋友解释扫描电子显微镜 (SEM): 朴素的语言、打合适的比方，便于他理解本质原理。

“把样品想成黑屋里的一小块月球地面。SEM 不用相机一次拍完整张照片，而拿一支看不见、细得不得了“电子铅笔”，一行一行扫过表面。铅笔尖碰到哪里，那里就会蹦出或弹回一些电子；旁边的“小桶”把它们接住，接得多就在屏幕上画亮一点，接得少就画暗一点。无数亮暗小点拼起来，我们就看见了肉眼看不到的表面纹理。

- ◆ 因果骨架没有丢：电子束逐点扫描 → 探测二次/背散射电子 → 信号强弱映射成像素。
- ◆ SEM 不是“拿更强的相机放大”，而是扫描、测量、重建；图像像三维地形，也不等于直接测得三维高度。
- ◆ 产品跃迁不只是“知道 SEM”，而是能建立听众模型：检索系统给术语，LLM 可以成为知识与人之间的认知接口。

图 11: GPT-4 为五岁孩子解释扫描电子显微镜所展示的知识重编码¹¹

这也解释了为什么 Agent 的上下文需要包含目标、受众、环境状态和历史反馈。缺少这些条件，模型只能生成表面合理的通用答案。

4.3 从 ChatGPT 到工具闭环

Transformer 带来可扩展训练，ChatGPT 把模型变成对话产品，chain-of-thought 与 test-time scaling 把更多计算放到推理时，检索和工具则把模型连接到外部状态。ReAct 类 Agent 的关键变化是交替执行“推理、行动、观察”，并在错误后恢复。

¹¹视频画面时间区间：00:22:00–00:22:15。

从 ChatGPT 到 Agent

反馈回路越来越长

Transformer 论文出现时，NLP 领域已经热起来了；但直到 ChatGPT，才出现“现象级”产品。

◆ Transformer 让计算可扩展；ChatGPT 把能力放进低摩擦的对话界面；reasoning 在测试时投入更多计算；Agent 再把环境反馈接回模型。产品跃迁来自整条链一起越过阈值。

- GPT o-family & DeepSeek R1 (2024-2025)
- Chain-of-thought 实现 in-context reasoning；test-time scaling 还可以增加轨迹、候选、搜索与验证，显著提高复杂推理能力
- GPT-5 配合 search，幻觉已经非常低；◆ 但风险也转移到了检索、来源选择和证据综合。
- 进化的 Claude family (2025)
- ReAct 型 Agent 从“不可用”到“能用”，忽然软件工程岌岌可危（这门课内容）
- 今天 Agent 产品开始爆发
- ◆ ReAct 的“推理—行动—观察”在 2022 年已有；2025 的突变，是模型可靠性、长上下文、工具接口和错误恢复共同越过可用阈值。

“

3 / 4

图 12: 从 ChatGPT 到 Agent：推理、检索、工具与错误恢复¹²

一个最小 Agent 循环可以表示为：

Listing 1: 可观察、可验证的 Agent 循环

```
1 state = load_goal_constraints_and_history()
2 while not done(state):
3     observation = observe_environment()
4     proposal = model(state, observation)
5     action = choose_tool_call(proposal)
6     result = execute(action)
7     evidence = verify(result, invariants, acceptance_tests)
8     state = record(state, action, result, evidence)
9     if evidence.failed:
10         state = revise_or_rollback(state)
```

4.4 Context 不再只是只读提示

当系统可以调用工具、接收观察并写回记忆时，context 变成了可写状态。Diffusion LM 可以同时修改多个位置；Agent loop 则把“模型状态、工具调用、环境观察、工作记忆”连接成 context-to-context 变换。真正困难的问题从“能否修改”转为“哪些约束必须保持不变”。

¹²视频画面时间区间：00:24:15–00:24:30。

不止 next token

Context 可以成为可写的状态

现在：世界上人才密度最大的领域，颠覆性的创新也许就在明天；Transformers 的上限还没到，上限更高的东西也肯定存在.....

- 为什么不是 $\Pr[\text{context}_{t+1} \mid \text{context}_t]$ 的模型呢？
- [Diffusion LM](#)
- Self-modifying of context...
- ♦ 从表达能力看，一串 next-token 已经能生成整个新 context: $P(x_{1:n} \mid c) = \prod_i P(x_i \mid c, x_{<i})$
- ♦ 自回归像用钢笔从左写到右；Diffusion LM 更像先铺开整份草稿，再多轮去噪，同时修改多个位置。
- ♦ Agent loop 已在系统层实现 context-to-context：模型读状态、调用工具、接收观察，再写回记忆。智能属于“模型 + context 更新规则”这个动态系统。
- ♦ Self-modifying context 把聊天记录变成可写的工作记忆；问题不只是“能否修改”，而是“什么必须保持为不变量”。

4/5

图 13: 不止 next token：把 context 视为可写状态¹³

使用 Agents：普通人视角

从“回答问题”到“把事情推进”

AI-native 的一代人，你真的应该用过。

- 课堂 demo：一个最小的 [Pi](#)
- DeepSeek harness 会在某节课专门讲（首批内测，于是有了一个梗图）
- ♦ 最小跃迁：AI 不只给出答案，还会观察结果、采取动作、继续推进。



1/5

图 14: 从回答问题到把事情推进的普通人 Agent 视角¹⁴

普通用户不必先学完全部模型知识才能使用 Agent，但必须学会把事情推进：不要只问“答案是什么”，而要明确下一步行动、可检查结果和失败后的处理。

¹³视频画面时间区间：00:27:30–00:27:45。

¹⁴视频画面时间区间：00:30:00–00:30:15。

形态	主要能力	缺失的工程保证
LLM	在给定 context 下生成下一 token	无外部状态、工具执行和独立验收
ChatGPT	多轮对话、指令跟随、面向用户的交互	会话之外的持续状态和可靠行动有限
Agent	推理、工具调用、观察、记忆和重试	权限、幂等、回滚、长程可靠性仍依赖 Harness
工程系统	明确状态、受控动作、监控、验收、审计和责任	需要持续维护，不能仅靠一次生成完成

4.5 想象力是委托边界

很多任务看似必须由人逐步操作，其实可以改写成“可观察、可检查、低风险”的数字动作。媒体从手抄本到印刷、从剪辑台到数字时间线的迁移说明，当表示和接口改变以后，人类往往先复制旧流程，再发明全新 workflow。

想象力是上限

你的**想象力**限制了 Agent 能完成的任务

- ◆ 新媒介出现时，人们通常先拿它复刻旧工作——早期印刷本甚至刻意模仿手抄本。
- ◆ 试着把“这事我不会”改写成：它能否成为一组可委托、可观察、可检查的行动？
- ◆ 更准确地说：对低风险、可检查的数字任务，今天的瓶颈常常是“没想到可以委托”。

图 15: 想象力限制了可以委托给 Agent 的可观察数字任务¹⁵

可委托不等于可放任

适合委托给 Agent 的任务应具有明确的数字边界、低成本回滚和独立验收信号。不可逆、高影响或法律责任重大的决策，需要人在关键节点停下来。

4.6 本章小结

LLM 提供条件生成，Agent 把生成接到工具、环境和记忆。能力的上限不仅由模型决定，也由人能否构造可观察状态、可验证动作和持续反馈决定。

¹⁵视频画面时间区间：00:32:00–00:32:15。

5 Agent 案例：把事情真正推进

5.1 案例一：一套现在运行的系统

Debian on USB 案例由 AI 生成安装、启动和编辑器配置，并在 QEMU 中验证。重点不是“AI 会装系统”，而是学习效率的转变：先用验证判断每一步是否正确，再在成功轨迹上理解原理，减少因机械配置造成的无反馈摩擦。

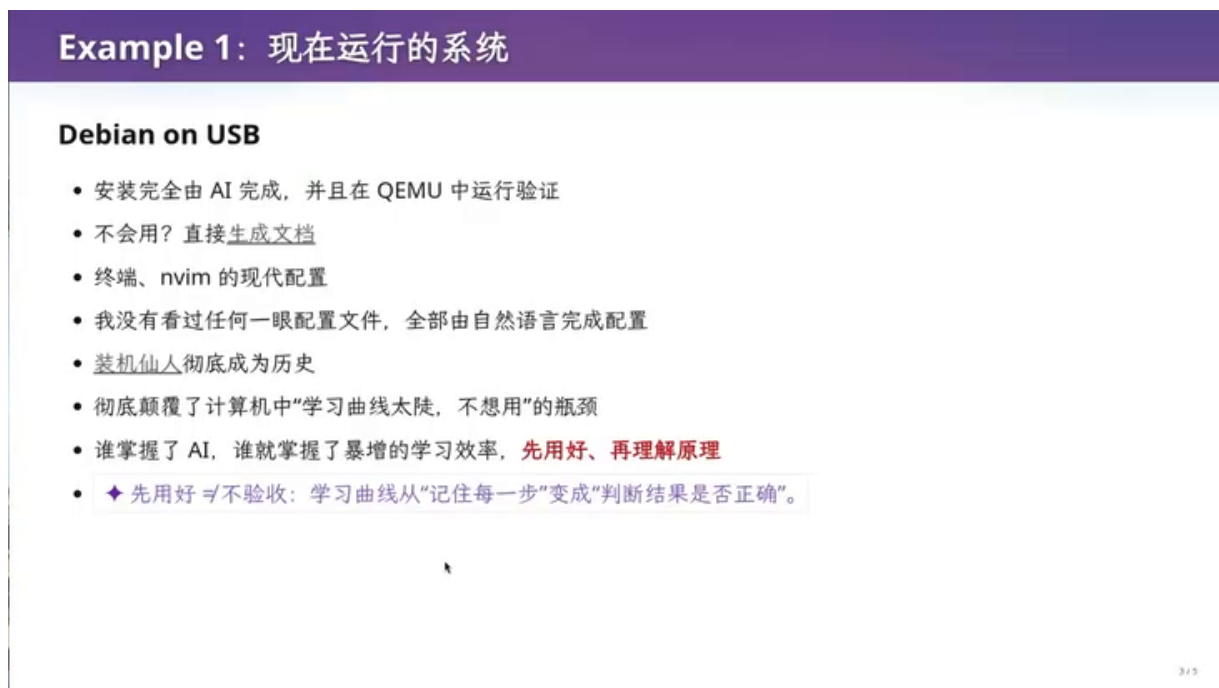


图 16: Debian on USB：AI 完成、QEMU 验证与配置生成¹⁶

这个顺序并不适合所有领域。若错误会造成数据丢失、权限泄露或真实设备损坏，必须先建立隔离环境、快照和回滚。AI 加速试错的前提是试错本身可控。

5.2 案例二：邮件与提醒形成闭环

邮件 Agent 读取历史回复，通过 db-query skill 检索相关往来，判断哪些邮件需要回复，并按照个人口吻生成草稿。Apple Reminders 不是单向通知面板，而是 Agent 与用户之间的双向接口：Agent 创建待办，人类确认、修改或完成，状态再回到下一轮决策。

¹⁶视频画面时间区间：00:32:15–00:32:30。

Example 2: 收发邮件写草稿

一个 db-query skill

- 包括了我历史上的所有回复
- 根据邮件内容自动归档；需要回复的，用我的口吻拟好草稿
- ♦ 核心资产不是“会写一封信”，而是能检索我过去如何判断、如何回应。

Reminders 是双向接口

- 通知推送到 Apple Reminders.app
- 我也可以通过 Reminders 给 Agent 发指令，而不是 Telegram 聊天框
- ♦ 于是形成闭环：收到邮件 → 检索历史 → 归档或起草 → 提醒人工确认。

4/5

图 17: 邮件数据库查询、回复草稿与 Reminders 双向接口¹⁷

5.3 案例三：深度阅读不是自动摘要

电子书 Agent 可以从古法学习的材料生成批注，再进一步追踪论证、定位原文、提出反例、重组知识并连接到个人笔记。简单摘要压缩信息，深度阅读则维护“结论、证据、反证、来源、个人问题”之间的结构。

Example 3: 读书、读论文

从古法学习到电子版 Agent

- 我又买了以前看过的书，重温一下古法学习
- 随后立即电子版 AI Agent 启动 😊
- Extension: 读 b 站，生成的讲义
- ♦ 生成讲义只是转换材料，不自动等于“深度阅读”。

AI 如何辅助深度阅读？

- ♦ 追踪论证：结论是什么？证据在哪里？每个回答都能跳回原文。
- ♦ 对抗阅读：让 Agent 提出反例、遗漏的前提，以及作者最强的反驳。
- ♦ 主动回忆：合上原文接受口试，再根据原文纠错：摘要不能替你理解。
- ♦ 重新组织：把新观点改写、归类，并连接到自己的旧笔记与真实问题。

5/5

图 18: 电子书 Agent 的深度阅读：追踪论证、对抗阅读与知识重组¹⁸

¹⁷视频画面时间区间：00:38:00–00:38:15。

案例	自动化对象	真正的验收信号
Debian on USB	安装、配置、启动	QEMU 可启动，关键工具可用，步骤可复现
邮件草稿	检索历史、分类、拟稿	引用正确上下文，语气符合用户，人工确认后发送
论文阅读	定位论证、生成批注、重组材料	每个结论可回原文，能给反证，能连接真实研究问题

5.4 共同模式：状态、动作、证据

三个案例的共同结构是：系统先获得可查询状态，再执行受限动作，然后用独立证据更新状态。若只保留最后的文本输出，就失去了判断系统是否可靠的最重要信息。

5.5 本章小结

Agent 的价值不是生成更多文字，而是把长期任务转化成有状态、有动作、有证据的闭环。不同领域的插件和验收器决定它是否真的能推进工作。

6 从白板 Agent 到完整生态

6.1 白板 Agent 只是起点

通用 Agent 可以完成前述任务，但完整生态还需要领域任务、领域插件和可验证接口。课程引用 Engelbart 的思想：增强智能来自“人、工具、语言和方法”的系统，而不是一个孤立的聪明部件。

¹⁸视频画面时间区间：00:40:30–00:40:45。

从白板 Agent 到完整生态

白板 Agent 只是起点

- 注意：以上任务，**白板 Agent** 就能完成
- Agent 拥有完整的生态系统！
- domain-specific 任务 → domain-specific 插件
- 计算机科学定律：你想做的事，一定也有别人想做
 - ✦ Engelbart 早在 1962 年就指出：增强智能来自“人 + 工具 + 语言 + 方法”的系统，而不是一个孤立的聪明部件 ([Augmenting Human Intellect](#))
- ✦ Agent 能力 $\approx$ 推理 $\times$ 可观测性 $\times$ 可操作性 $\times$ 反馈

1/5

图 19: 从白板 Agent 到领域插件与完整能力生态¹⁹

可以把 Agent 能力粗略写成：

$$A \approx R \times O \times U \times F$$

- R 表示推理与规划能力；
- O 表示环境的可观察性；
- U 表示可操作性与工具覆盖；
- F 表示反馈的及时性与可靠性。

乘法形式强调任何一项接近零都会卡住整个系统；它是分析框架，不是课程给出的经验定律。

6.2 浏览器不是黑盒

Computer Use 经常被误解为“模型看到像素并点击”。计算机专业学生还有更稳定的控制面：DOM、网络请求、事件、状态机和 Chrome DevTools Protocol。像素适合兜底，结构化接口更容易验证和复用。

¹⁹视频画面时间区间：00:50:30–00:50:45。

浏览器不是黑盒

Browser Use / Computer Use

- Browser CDP (Chrome DevTools Protocol)
- DOM Query、Network、模拟用户输入.....
- 这是 CS 学生最大的优势：对“计算机世界里有什么”有概念
- ~2020s，我们做过一个很疯狂的 project：在 OpenJDK 上运行 Android app，用 CDP 渲染界面
 - ✦ 普通人看到像素和按钮；CS 学生看到 DOM、协议、事件和状态机——系统里往往藏着比“点鼠标”更稳定的
- 控制面
 - ✦ Domain-specific 插件的本质：把巨大、模糊的动作空间，压缩成语义明确、容易验证的操作

2 / 5

图 20: Browser Use 和 Computer Use 不应被视为黑盒²⁰

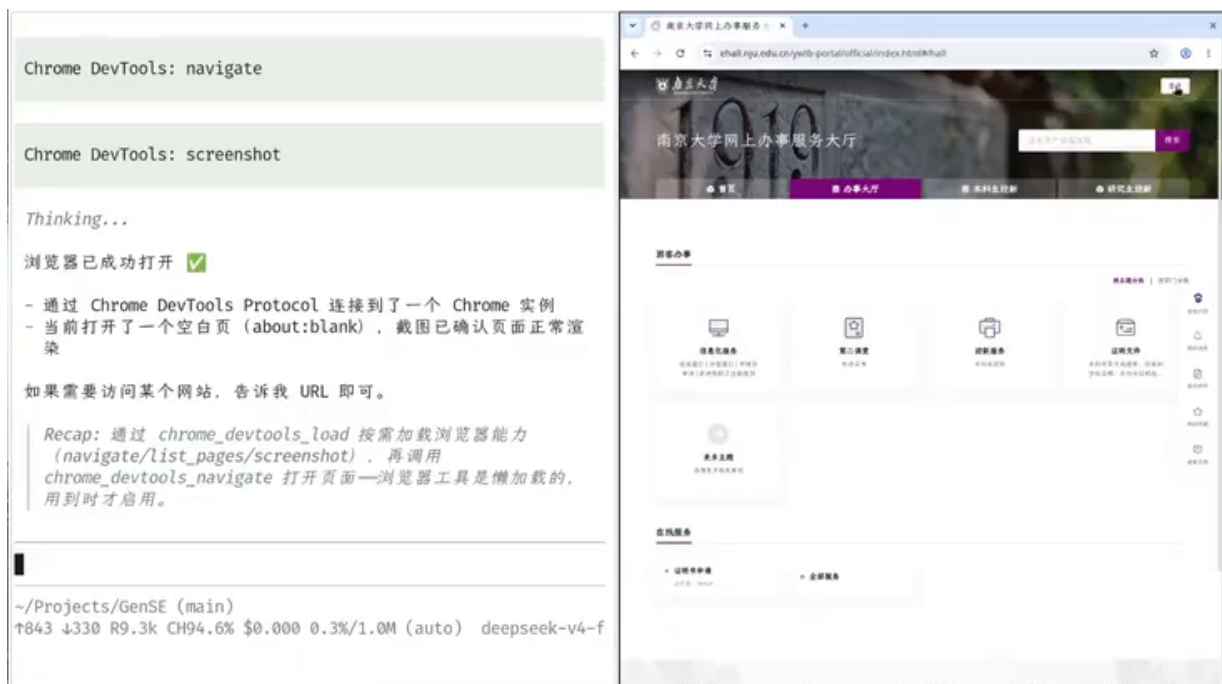


图 21: 通过 Chrome DevTools Protocol 导航南京大学网页的现场闭环²¹

现场演示通过 CDP 打开空白页、导航到南京大学网上办事服务大厅，再观察页面是否正确加载。闭环不止是 ‘navigate’ 调用成功，而是浏览器状态与目标页面均提供可检查证据。

²⁰视频画面时间区间：00:51:45–00:52:00。

²¹视频画面时间区间：00:55:15–00:55:30。

6.3 从一次操作到软件资产

让 Agent 观察页面、理解流程并完成一次任务只是 demo。更高价值的做法是把成功轨迹沉淀为脚本或插件，再进行校验、修改和重放。Programming by Demonstration 的思想因此重新重要：自然语言负责提出目标，结构化轨迹负责形成可执行资产。



图 22: Programming by Demonstration：把一次操作沉淀成可复用资产²²

6.4 能力组合与 workflow 自举

信息源、信息处理和操作可以自由组合。网页、B 站、arXiv 或 Hacker News 提供信息；模型完成抽取、比较和规划；外部系统完成下单、上传或记录。若整个过程被观察和记录，Agent 还可以发现重复模式并生成下一层自动化。

²²视频画面时间区间：00:59:30–00:59:45。

自由组合，无限复用

信息源 → 信息处理 → 操作

- 刚才所有的能力都可以互相增强
- 信息源（网页、B 站、arXiv、Hacker News.....）+ 信息处理 + 操作（淘宝下单、Box 上传、iCloud.....）
- 于是可以理解为什么 OpenClaw 和 Hermes 是现象级产品了
- ♦ 组合的关键不是插件够多，而是接口足够窄、稳定；这正是 Unix pipe 留下的软件工程遗产

workflows 也能自举

- 我们完全可以录屏一天，让 AI 自动改进流程
- ♦ 观察操作 → 发现重复模式 → 生成自动化 → 回放验证 → 人确认 → 收集新异常

4/5

图 23: 信息源、信息处理与操作的自由组合及 workflows 自举²³

6.5 Harness 的可靠性债务

能做不等于能放心托付。若单步成功率为 p ，连续 n 步全部成功的朴素上界为：

$$P_{\text{all}} = p^n$$

当 $p = 0.99$ 、 $n = 100$ 时：

$$0.99^{100} \approx 0.366$$

也就是说，单步 99% 看似很高，百步任务无故障完成率却只有约 36.6%。长流程需要状态、权限、重试、幂等、回滚和验证，而不能只依赖模型“再想一下”。

²³视频画面时间区间：01:00:00–01:00:15。

未来已来，Harness 未稳

能做，不等于能放心托付

- OpenClaw/Hermes 预见这样的未来
- 可惜现在的模型/Harness 还没有完全准备好，体验并不好
- ◆ 单步成功率即使是 99%，连续 100 步全成功也只有 $0.99^{100} \approx 36.6\%$ ；长流程需要状态、权限、重试、验证和故障恢复

一个刺耳的问题

- 我很难理解旧人类为什么还能心安理得地每天做重复的事情
- ◆ 稳定、可描述、可验证的重复，本质上就是一个尚未被抽象出来的程序

5/5

图 24: Harness 尚不稳定：单步 99% 在百步任务中仅约 36.6%²⁴

长程 Agent 的可靠性清单

- **状态**：重启后能否知道做到哪一步，是否存在重复执行风险；
- **权限**：读、写、删除、发送、付款等动作是否按最小权限分离；
- **幂等**：网络超时后重试会不会重复提交或破坏外部状态；
- **验证**：成功信号来自工具自己的返回值，还是独立观察和验收器；
- **恢复**：失败能否回滚，不能回滚时是否先使用沙箱或影子模式；
- **升级**：何种不确定性、金额、影响范围或重复失败必须交还给人。

6.6 CS Professional：把需求编译成现场工具

Agent 不应只用于交作业。计算机专业的优势是知道可编程性的边界：即使没有正式 API，只要存在可控输入、可观察输出和可判定完成条件，就可以把一次性自然语言需求编译成可重复执行的工具。

²⁴视频画面时间区间：01:04:15–01:04:30。

使用 Agents：CS Professional 视角

别把 Agent 只用来交作业

- 最后，我们是 Computer Science 的学生，非常清楚地知道**编程**的能力界限在哪里
- 用 Agent 做作业实在是太大材小用了
- ♦ 1979 年 VisiCalc 让会计人员不必排队等程序员：能描述流程的人，第一次可以直接拥有工具。

把需求“编译”成工具

- ♦ Agent 的真正杠杆是**现场制造工具**：把一次性的自然语言需求，变成可重复执行的闭环。
- ♦ 可编程性的边界不在“有没有 API”，而在：能否找到**可控输入、可观测输出、可判定的完成条件**。

1/5

图 25: CS Professional 视角：把自然语言需求编译成现场工具²⁵

从聊天到工程系统的升级条件

对话给出建议；工具改变状态；工程系统还必须记录状态变化、验证不变量、管理权限，并在失败时提供恢复路径。

6.7 本章小结

完整 Agent 生态由模型、领域插件、结构化控制面、反馈和可靠性机制共同构成。Harness 的主要工作不是让模型更会说，而是把开放动作空间压缩为明确、可验证的操作。

7 现场造工具：边界、协议与证据链

7.1 考试统分：不是 OCR，而是证据链

课堂中的统分工具采用四阶段流水线：拍照并离线识别身份与小分；按题目顺序录入并检查总分；写入教务系统 Excel 模板；最后由 Agent 逐份校对。这种设计保留原始照片、小分中间表示和 Excel 派生结果，便于反向 readback、定位错位与发现漏项。

²⁵ 视频画面时间区间：01:05:15–01:05:30。

现场造工具：考试统分

四阶段流水线

- Phase 1: 按下回车，拍摄一张照片，Offline qwen3.8-27b 识别姓名、学号、小分
- Phase 2: 按照反序，按下回车报出 [姓名][学号][11+12+13+14+15=65]，我检验后誊写总分
- 分数自动汇总到教务系统的 Excel 模板
- 最后，Agent 直接登记分数 (日常应用配合豆包输入法更佳)

不是 OCR，而是证据链

- ✦ 照片是原始凭证，小分是中间表示，总分与 Excel 是派生结果。
- ✦ “反序 + 报算式”像航空管制的 readback：用不同表示制造冗余，专门截获错位、漏项和误识别。
- ✦ 人只驻守最窄的高风险接口：让机器承担吞吐量，用一次复核阻止错误进入教务系统。

2/5

图 26: 考试统分的四阶段证据链，而不是一次 OCR²⁶

“不是 OCR”强调系统目标不是识别几个数字，而是建立证据链。OCR 只是局部组件；总分一致性、行列映射、异常样本和最终人工确认共同决定系统是否可信。

7.2 只进不出的机器也可能泄露

假设隔离机允许输入文件，却禁止运行不可信程序、访问网页或输出文件。只要屏幕像素仍可观察，信息通道容量就不是零：Agent 可以把文件编码成二维码动画，再由外部相机恢复。

²⁶视频画面时间区间：01:06:00–01:06:15。

Quiz: 只进不出的机器

怎么偷出文件？

- Model: 文件只进不出。不能执行不信任的程序、不能显示不信任的网页；可以远处悄悄摄屏，但复杂文件就有点麻烦。
- ♦ 安全边界限制了“文件”，却没有限制**信息**：只要像素仍可观察，出口容量就不为零。



图 27: 只进不出的机器仍可经可观察像素泄露信息²⁷

这个 Quiz 的意义不是鼓励绕过安全，而是训练边界思维：安全策略限制了“文件”，却没有限制“信息”。工程审计必须追问所有可观察通道，而不是只检查名义接口。

7.3 Excel、二维码和鼠标组成协议栈

解决方案把 Excel 视为可编程显示设备：公式生成 QR Code，鼠标抖动作为输入线路，二维码作为发送端，外部程序通过 Raw Socket 实现协议。Reed-Solomon、纠错、ACK/NAK 和重传使二维码不再只是图片，而成为可靠传输介质。

²⁷视频画面时间区间：01:09:30–01:09:45。

把远程桌面变成内网

My Solution

- `qrngen.xlsx`: 我可以用 Excel 公式算 QRCode :)
- Rx = Mouse (抖鼠标)
- Tx = QRCode (QrGen)
- 一个 Raw Socket 上的协议栈

“网卡”只是网络的一种实现

- ✦ QRCode 已经提供同步与 Reed-Solomon 纠错；再加帧号、校验和、ACK/NAK、重传，就能传复杂文件。
- ✦ 最关键的重解释：Excel 是**可编程显示设备**，鼠标是**输入线路**；协议只需要能承载符号的媒介。

4/5

图 28: 用 Excel、QR Code、鼠标和 Reed-Solomon 构造单向协议栈²⁸

7.4 Everything Is Code: 从 Bits 到 Atoms

视频可以拆为脚本、分镜、素材、时间线、字幕和渲染流水线；论文可以拆为问题、证据、实验、图表和可复现构建；实体制造可以拆为 CAD、仿真、BOM、固件和 G-code。它们的共同点是把结果表示为可修改的控制描述，再由渲染器、排版系统或机器负责兑现。

Everything Is Code

Agent 可以造 Everything

- 视频
- ✦ 脚本、分镜、素材、时间线、字幕、渲染流水线
- 论文
- ✦ 问题、证据、实验、图表、可复现构建
- 东西 (VibeFab)
- ✦ CAD、仿真、BOM、固件、G-code: 最后由机器把程序“打印”进现实

从 Bits 到 Atoms

- ✦ 从 Jacquard 提花机的穿孔卡到 CNC/3D 打印: 可制造物先变成可复制、可修改的控制描述。
- ✦ 视频、论文、零件只是不同后端——Agent 生成中间表示，渲染器、排版系统和机床负责兑现。

5/5

图 29: Everything Is Code: 从视频和论文到 CAD、BOM 与 G-code²⁹

²⁸视频画面时间区间: 01:12:30–01:12:45。

“一切皆代码”的边界

可编码不代表无成本。物理世界有材料、时间、安全与不可逆性；组织世界有权限、伦理与责任。表示层越强，越需要独立验证它和现实之间的对应关系。

7.5 本章小结

工具的价值来自边界建模和证据链，而不是调用了多少模型。把对象转成代码后，工程师仍要验证表示、传输、执行和现实结果之间没有断裂。

8 qrgen 失败案例：跑出来不等于做成

8.1 产品是系统性工程

“AI 造航母”式乐观忽略了端到端可靠性。产品包含前端、后端、交互、运维、组织和数据等多个环节；任一环节失败都可能暴露整个系统。若十个环节独立且各有 90% 可靠性，整体成功率为：

$$P_{\text{system}} = \prod_{i=1}^{10} p_i = 0.9^{10} \approx 0.349$$

Wow! AI 造航母？

“就差一个……”

那我直接让 AI 造航母不就行了？不行。Pre-LLM 时代就有的老梗：

- idea 有了，就差一个程序员
- ABCDE 轮都有了，就差一个天使投资人

产品是系统性工程

- 前端、后端、交互、运营……任何一个环节做不好，产品都会暴毙
- 虽然 AI 都可以做，但组织不当（让 AI 直接接管组织）也会暴毙
 - qrgen.xlsx 就直接暴毙了；ehall.nju.edu.cn 已经暴毙

- ◆ 产品完成度是乘法：若 10 个环节各有 90% 可靠，整体成功率只有 $0.9^{10} \approx 35\%$ 。AI 提高单点产能，不会自动消除接口与协作风险。

1/6

图 30: 产品是系统性工程：十个 90% 环节的整体成功率约为 35%³⁰

实际环节通常并不独立，因此这个计算只是提醒：局部漂亮不能替代系统验收。更复杂的依赖可能使尾部风险更难发现。

²⁹视频画面时间区间：01:13:45–01:14:00。

³⁰视频画面时间区间：01:14:45–01:15:00。

8.2 一句明确需求里的隐藏规格空间

qrigen 的表面需求很具体：使用 Excel 2019 兼容公式，把任意长度输入编码为 Base64，再生成逐帧二维码动画，用 F9 刷新，每秒一帧，扫描后拼接并恢复原文件。但“任意字符串”同时隐藏编码、分片顺序、容量、空串、超长输入、刷新时钟和工作表膨胀等选择。

理解 qrigen 的暴毙

一个“明确”的需求

“生成 qrcode.xlsx，只使用 Excel <= 2019 兼容公式，把输入的一个任意字符串（长度可能达到数千），先用 base64 编码，然后转换成 QRCode 的动画：每一帧是 base64 编码的一个片段，F9 显示刷新，1 秒一帧，循环播放；二维码中的数据拼接后应该是完整的 base64 编码字符串。这个任务很容易失败，用接近真实的设定（脚本）验证公式能正确计算。

隐藏的规格空间

- ◆ 一句“任意字符串”同时留下编码、分片顺序、容量边界、空串/超长输入、F9 重算和计时语义等选择：需求很短，不等于规格空间很小。
- ◆ 真正的验收式不是“文件能打开”，而是 $\forall s : \text{Base64}^{-1}(\text{concat}(\text{scan}(\text{frames}(s)))) = \text{bytes}(s)$ 。

2 / 6

图 31: qrcode.xlsx 的明确需求与隐藏规格空间³¹

真正的验收不是“文件能打开”，而是：

$$\text{Base64}^{-1}(\text{concat}(\text{scan}(\text{frames}(s)))) = \text{bytes}(s)$$

- s 是任意合法输入；
- frames 生成有序二维码帧；
- scan 是独立解码器；
- 等式要求端到端字节完全一致。

8.3 跑出来不等于做成

GLM-5.3 在 25 分钟思考和超过 32K 输出后，用约六小时给出结果；DeepSeek V4 Pro 最终给出硬编码版本并承认设计需要重想。两者都可能生成可以打开的 Excel，却留下 2MB 级 AI slop、硬编码公式或不可维护结构。

³¹视频画面时间区间：01:17:15–01:17:30。

跑出来，不等于做成

结果

- GLM-5.3: 第一次思考就有 25 分钟的 CoT, 然后超过 32K output token max; 6 小时终于完成
- DeepSeek V4 Pro: 已经无法遵循指令, 最终给了一个硬编码的版本
- 骂过以后: "This is a fundamental design flaw. I need to rethink."
- 但是: **都是完全无法维护的 AI slop**, 2 MB 大小的 Excel

同一种死法

- 和 ehall.nju.edu.cn 完全一样: 上来就对着需求写, 直接陷入维护泥潭
- ♦ 硬编码把 $\forall x$ 的程序问题, 偷偷换成了“让这个 x_0 通过”: 它可以演示成功, 却没有实现需求。
- ♦ AI slop 的本质不是文件大, 而是改动没有局部性: 一个需求变化, 就迫使整张表重新生成: 2 MB 只是症状。

图 32: 程序跑出来不等于做成: 硬编码、AI slop 与不可维护性³²

失败不在于模型没有继续输出, 而在于架构没有表达任务的不变量。继续追加 Prompt 只会把工作表和上下文一起膨胀。

8.4 少量 Prompt, 换一个架构

更好的转折不是“再多想一会”, 而是从“生成一个能跑的文件”切换到“设计可复用的计算结构”。公式模板和工作表结构保持 $O(1)$, 只有输入数据、计算量和帧数随长度 $O(n)$ 增长。

³² 视频画面时间区间: 01:18:45–01:19:00。

少量 Prompt, 换一个架构

对比

- 仅通过少量 prompt, GLM-5.3 在 200K 上下文内完成第一个正确版本
- 工作表不因为文本长度膨胀
- 公式可复用、代码可维护

关键不是“再多想一会”

- ✦ 真正的转折, 是从“生成一个能跑的文件”切换到“设计一个可复用的计算结构”。
- ✦ 相对文本长度, 公式模板和工作表结构应保持 $O(1)$; 只有数据、计算量和帧数按 $O(n)$ 增长。
- ✦ 架构检查题: 输入从 1 KB 变成 10 KB 时, 究竟有哪些东西必须增长?

图 33: 少量 Prompt 换架构: 从生成文件转向设计可复用计算结构³³

架构检查应问: 输入从 1KB 变成 10KB 时, 哪些资产必须增长? 哪些模板和公式理应保持不变? 如果静态结构随数据量线性复制, 系统已经把数据和程序混在一起。

8.5 Intent、Spec 与 Impl 三个空间

讲者把问题拆成三个空间: Intent Space 是需求方也未必完全知道真实意图; Spec Space 是从意图提炼出的明确约束; Impl Space 是计算机程序可以接受的无歧义指令。

³³视频画面时间区间: 01:21:15–01:21:30。

Intent、Spec 与 Impl

三个空间

- Intent Space: 需求的提出者也不 100% 确定他想要什么, 比如“给我弄一个国民级的应用”
- Spec Space: 从 Intent 到 Spec, 已经曲解了很多
- Impl Space: 计算机程序只接受 100% 无歧义的指令



“正确”发生在哪一层?

- $\text{Impl} \models \text{Spec}$ 只说明“按规格正确”, 并不推出“满足意图”: 测试可以证明前者, 却无法替人决定后者。

5/8

图 34: Intent、Spec 与 Impl 三个空间以及正确性所在层次³⁴

空间	典型内容	主要失败
Intent	为什么要做、谁使用、风险和价值	人自己也没有想清, 目标随反馈变化
Spec	接口、约束、不变量、验收与边界	只覆盖可见样例, 遗漏真实约束
Impl	数据结构、算法、代码和部署	正确实现了错误或不完整的 Spec

测试可以证明实现满足某些 Spec, 却不能自动证明 Spec 满足真实 Intent。这个 gap 是需求工程、软件工程与人机协作的核心。

正确性因此至少有三层。第一层是**执行正确**: 代码没有崩溃并产生某种输出。第二层是**规格正确**: 对所有已定义边界, 输出满足不变量和验收条件。第三层是**意图正确**: 规格本身解决了用户真正的问题, 且没有违反未写入测试的安全、兼容、维护和责任约束。大量 AI demo 只达到第一层, 却在界面上制造“已经完成”的错觉。

测试通过的有限含义

测试是对实现的有限观察, 不是真实意图的完整副本。若测试样例、奖励或 benchmark 可以被直接优化, 系统可能学会满足 proxy 而绕开目标。工程师必须维护测试来源、独立 oracle、未见样例和现实使用反馈。

8.6 模型以猜测补齐 gap

模型常通过猜测补齐未说明细节; 用户又可能厌烦无穷追问。正确策略不是让系统猜得更自信, 而是把关键不确定性变成低影响、可逆、可验证的实验。课程给出一个决策条件:

³⁴ 视频画面时间区间: 01:24:15–01:24:30。

$$P(\text{猜错}) \cdot C(\text{返工}) > C(\text{澄清})$$

当左侧更大，应停下来提问；否则先做最小实验，以实际反馈缩小 gap。

模型如何补 Gap?

猜

- 模型补 gap 通过的是猜——如果事无巨细都问，人会觉得很讨厌
- 有无穷多 spec“满足”intent，也有无穷多 impl 方法“满足”spec
- 模型训练的目标是“在有限的 token 内完成任务”，太容易“立即遵循指令”；正确的软件工程师动作：**先别急，再讨论一下**
- 一旦在重大决策上犯错，就需要巨大的成本来补；它们不像人一样有重构意识 😊

人机协同的决策题

- ◆ 目标不是消灭猜测，而是让猜测显式、可逆、可验证：低影响决策先猜后验；高影响、难回滚的决策先停下来问。
- ◆ 当 $P(\text{猜错}) \cdot C(\text{返工}) > C(\text{澄清})$ 时，应当提问；否则先做最小实验。
- **模型/Agent 很强**，但如何人机协同发挥最大的能力？这就是这门课真正想讨论的

6 / 6

图 35: 模型以猜测补齐 gap，人机协同要把决策变成低成本实验³⁵

8.7 本章小结

qrigen 的教训不是某个模型不够强，而是自然语言需求中存在巨大规格空间。工程进步来自不变量、独立验收、可复用架构和快速反馈，而不是更长的生成。

9 软件工程为什么仍然必要

9.1 焦油坑来自耦合

《人月神话》与《死亡搁浅》的焦油坑比喻强调：局部功能看起来简单，真正成本来自耦合、隐含知识和验证链。增加一点功能可能要求大量 token、重测和上下文同步，因为约束是全局的。

³⁵视频画面时间区间：01:29:00–01:29:15。

“The Mythical Man-Month”

大型动物一旦陷入焦油坑就很难逃脱，而濒死的动物又会引来各种掠食者。



- 也是《死亡搁浅》中的原型
- qrgen 已经进入焦油坑：增加一点功能，都需要消耗巨额的 token

✦ 焦油不是代码量，而是耦合、隐含知识和验证债：功能是局部的，约束却是全局的。

1 / 2

图 36: 人月神话：焦油坑来自全局耦合，而不是局部代码量³⁶

软件工程处理的不是单个函数，而是多人、工具、接口、历史和责任构成的系统。模型降低局部实现成本，反而使系统级协调成为更大比例的工作。

9.2 软件工程研究什么

软件工程研究怎样在人和工具的共同协作下做成大事。领域既包含 AI4SE，也包含 Analytics、Architecture and Design、Dependability and Security、Evolution、Human and Social Aspects、Requirements and Modeling、Testing and Analysis 等方向。

³⁶视频画面时间区间：01:30:45–01:31:00。

软件工程研究什么？

怎么把一件大事，在**人和工具的共同协作**下做成？

- 有技术，也有人类学、社会学、管理学的部分（人因）
- [ICSE 2027 Call for Papers](#): AI4SE, Analytics, Architecture and Design, Dependability and Security, Evolution, Human and Social Aspects, Requirements and Modeling, SE4AI, Testing and Analysis
- 教育体系“考试就是终点”的弊端
- 所有“考题”都是分钟级能求解的；大作业可以摸鱼
- 没有 self-motivation，就从未训练过如何解决 long-horizon 的问题

✦ 工程能力，是在稀疏、延迟的反馈下仍能维持方向；软件工程因此同时组织技术、协作关系、人的动机和时间。

2/8

图 37: 软件工程研究人和工具在共同协作下做成大事³⁷

教育体系若只训练分钟级、终点明确的考题，就无法训练 long-horizon 能力。工程能力需要在稀疏、延迟的反馈下保持方向，并处理组织技术、协作关系、人的动机和时间。

9.3 LLM 时代，失败的定义没有变

软件失败仍然是产物没有满足真实 Intent 和 constraints。真实意图和真实 Spec 难以获得；长程奖励信号较弱；失败项目不能通过一次推理重新生成；利益相关方也不会为“模型很努力”买单。

³⁷视频画面时间区间：01:32:00–01:32:15。

LLM 时代，失败的定义没有变

失败 = 软件不满足 **intent** 和 **constraints**

- “真实 intent”和“真实 spec”都非常困难，gap 的自由度很高
- GRPO 的 long-horizon reward 信号其实很弱
- 不像人在经历项目失败以后会“三观尽毁”再重生（方法论重塑）
- 我相信这个问题会在不久的将来被“大力出奇迹”解决
- 这样人类就不剩下什么了

◆ spec 不是真相，只是对 intent 的一次有损观测；测试和 reward 看见的常常只是末端 proxy——“能通过测试”不等于“满足真实意图”。

3/3

图 38: LLM 时代失败的定义没有变：不满足真实 intent 与 constraints³⁸

测试与 reward 通常只看见末端 proxy。能通过测试不等于满足真实意图，正如作业完成不等于学习发生。Agentic 系统需要维护代理指标与真实目标之间的证据联系。

9.4 让失败更小、更早、更可逆

避免工程失败的核心不是消灭所有错误，而是把不可控的大赌注改造成可验证的小赌注：拆解、独立验证、持续集成、门控、短迭代和快速回滚构成反馈控制环。

³⁸视频画面时间区间：01:36:00–01:36:15。

怎么让工程不失败？

把不可控的大赌注，改造成可验证的小赌注

- 拆解：把大“国民级应用”的大 gap 拆成小 gap
- 验证：测试、CI，确保门控
- 迭代：小步提交、快速验证，让噪音尽早暴露
-

◆ 这不是三条技巧，而是一个反馈控制环：拆解缩短因果距离，门控限制错误传播，迭代提高反馈频率；目标不是零错误，而是错误出现得早、炸得不远、回退得便宜。

4 / 8

图 39: 把不可控的大赌注拆成可验证的小赌注³⁹

qrgen 的开发顺序应按“风险退休”安排：先冻结 Excel 版本、Base64 可重组、每帧可解码和工作表不随输入膨胀等不变量；再建立独立 oracle 和 golden cases；随后依次增加单帧、分片、多帧与刷新；性能和兼容性最后加入，每一步都保留回归门。

Discussion：如何安排 qrgen 的开发？

按“风险退休”的顺序，而不是按最终需求堆功能

- ◆ 先冻结 invariants: Excel ≤ 2019、Base64 可重组、每帧可真实解码、工作表不随输入长度膨胀
- ◆ Stage 0: 先写独立 oracle，用随机、边界和 golden cases 验证结果
- ◆ Stage 1: 固定短字符串 → 单帧 QR → 被真实解码器还原
- ◆ Stage 2: 任意输入 → Base64/分片正确：仍只选择一帧
- ◆ Stage 3: 多帧顺序、循环、拼接可自动验证
- ◆ Stage 4: 最后加入 F9/定时、兼容性和性能：每一阶段都留下可执行回归门

5 / 8

图 40: 按风险退休次序推进 qrgen，而不是按最终需求堆功能⁴⁰

³⁹视频画面时间区间：01:38:00–01:38:15。

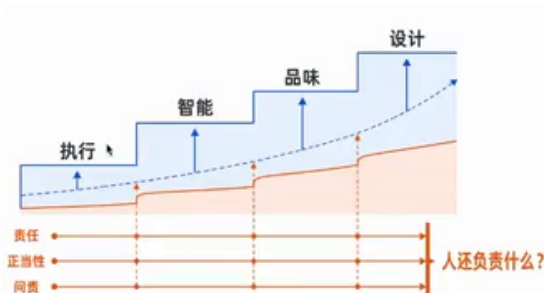
⁴⁰视频画面时间区间：01:38:15–01:38:30。

9.5 人的稀缺性继续向上游迁移

当 AI 能生成代码、执行操作甚至模仿品味时，人的稀缺性从亲手生产迁移到选择“什么值得存在”、设计怎样验证、决定何时停下以及承担后果。正确性和问责不会因为能力迁移而自动迁移。

当 AI 连“品味”也学会以后？

人的稀缺性继续向上游迁移



- ◆ 从亲手生产，转向选择“什么值得存在”
- ◆ 能力可以迁移；正当性与问责不会自动迁移
- ◆ 真正困难的问题也许不再是“怎么做”，而是“为何做、为谁做、后果由谁承担”

8 / 8

图 41: AI 会品味之后，人的稀缺性继续向选择、设计与责任上游迁移⁴¹

软件工程暂时还没有死

为什么要招你，而不是买一个 \$200 的 Claude Max？

我们有一个前所未有的机遇：一个人能像以前一个团队一样解决现实世界中的问题——保研的时候，希望大家能答上这个问题，除了你可以坐牢？

- ◆ 不要和 Agent 比产能。你的稀缺性是：方向 × 验证 × 责任——让一支可复制的 Agent 团队朝真实结果收敛。
- ◆ “可以坐牢”只点出了责任；责任必须和最终决策权、情境判断与停止权绑定。没有这些权力却只负责背锅的人，只是自动化系统的 moral crumple zone。
- ◆ 30 秒回答：我能找到值得解决的问题，把未知拆成便宜的实验，用证据让坏方向及时停下，并对长期结果负责。

2 / 2

图 42: 软件工程暂时还没有死：方向、验证、责任与长期结果仍不可外包⁴²

⁴¹视频画面时间区间：01:38:30–01:38:45。

⁴²视频画面时间区间：01:39:00–01:39:15。

为什么不是买一个更贵的模型就够了

方向、验证和责任需要最终决策权。可以工作交给 Agent，但不能把责任隐藏在自动化系统的缝隙里。真正可靠的团队让 Agent 可复制、可审计、可中止，并由人对长期结果负责。

9.6 本章小结

软件工程暂时没有死，因为系统失败的根源不是打字速度，而是意图不完整、全局耦合、证据不足、反馈延迟和责任不清。AI 改变实现方式，却放大了这些经典问题的重要性。

10 总结与延伸

10.1 讲者的结论

本讲从能力日用品化出发，说明传统作业和简历为何失去鉴别力；再用 LLM、Agent、Computer Use、Debian、邮件、论文阅读、浏览器和 Excel 二维码展示新的可编程空间；最后通过 qrgen 失败案例回到软件工程：实现跑起来只是开始，真实目标、约束、验证和责任仍决定系统是否成功。

10.2 本笔记的结构化压缩

可以用一条链压缩全讲：

Intent → Spec → Impl → Execution → Evidence → Feedback → Revised Intent

每个箭头都可能丢失信息。Agent 提高了生成和执行速度，却不能消除这些映射误差；软件工程的任务就是让误差尽早暴露、让反馈尽快返回、让责任始终可定位。

10.3 五个可执行结论

1. 不要用“模型能否生成”定义问题，先定义成功证据与不可违反的不变量。
2. 把一次成功操作沉淀为可重放工具，并为它补上状态、权限、验证和回滚。
3. 先做能够退休最大风险的最小实验，不要按最终界面从外到内堆功能。
4. 在作品中保留版本历史、失败与决策，使能力不再依赖一份可伪造的最终结果。
5. 把人的时间放到上游：问题选择、架构、验收、伦理和责任，而不是和模型竞争机械执行。

10.4 开放问题

- 如何设计既允许 AI、又能真实训练能力的课程评价？
- 如何给长程 Agent 建立跨工具、跨天甚至跨团队的可靠证据链？

- 当 Intent 会在反馈中改变时, Spec 应如何版本化并与责任绑定?
- 哪些人类判断必须保留最终决策权, 哪些可以在充分验证后委托?
- 当一切都可编码时, 如何避免现实世界的成本、伦理和不可逆性被表示层遮蔽?

10.5 资料与证据边界

- 课程视频: <https://www.bilibili.com/video/BV1pb8o6yE8f/>
- 课程主页: <https://jyywiki.cn/GSE/2026/>
- 本笔记中的时间脚注来自 15 秒密集采帧与 Whisper 时间轴对齐。
- 课程页面当前提供课程定位和主题规划; 本讲的详细论证、案例与数值以视频证据为准。

10.6 本章小结

“欢迎来到未来”不是宣告工程师被替代, 而是提醒工程师改变工作重心: 让模型负责廉价执行, 让系统暴露可检查状态, 让反馈驱动下一步, 让人对方向和后果负责。